Guion de presentación — Avances del proyecto BOOKLY

Materia: Computación Móvil
Tipo de sesión: Avances de proyecto
Duración orientativa: 10 minutos
Formato: 3 ponentes
Enfoque: lógica de negocio de la aplicación y funcionamiento del frontend móvil Android

> **Objetivo de la presentación:** explicar de forma clara qué resuelve BOOKLY, cómo funciona la lógica de negocio mediante la API y cómo el frontend se convirtió en una aplicación móvil funcional, sin profundizar en detalles técnicos innecesarios.

Distribución del tiempo y responsables

Ponente	responsibility	Tiempo
Ponente 1	Contexto, alcance y avances generales	3 min
Ponente 2	Lógica de negocio y funcionamiento de la API	4 min
Ponente 3	Tecnologías, frontend móvil, pruebas, resultados y cierre	3 min

Guion de exposición

Diapositiva 1 — Portada — Ponente 1 — 30 segundos

En pantalla

- **BOOKLY**
- Avances del proyecto
- Computación Móvil
- Fecha de entrega / periodo 2026-II

Guion

> “Buenos días. Somos el equipo de BOOKLY y hoy vamos a presentar los avances de nuestro proyecto de Computación Móvil. BOOKLY es una aplicación orientada a la gestión de citas para salones de belleza y barberías. En esta oportunidad explicaremos qué hemos construido, cómo funciona la lógica de negocio y cómo conseguimos llevar el frontend hasta una aplicación Android instalable y firmada. La exposición tendrá aproximadamente diez minutos y la presentaremos entre los tres integrantes.”
>
> **Nota del equipo:** evitar decir “API REST” de forma constante. Hablar de “la API”, “el sistema central” o “el servicio del proyecto” aporta mayor claridad sin perder precisión.

Diapositiva 2 — Problema y propuesta de valor — Ponente 1 — 1 minuto

En pantalla

- **Problema:** coordinar citas manualmente entre cliente, profesional y administración.
- **Propuesta:** catálogo de servicios, disponibilidad real, agenda y reservas desde el celular.
- **Roles:** administrador, profesional y cliente.

Guion

> “El problema que identificamos fue la gestión manual de las citas. Un cliente debía contactar al salón, el profesional debía coordinar su tiempo y el administrador no tenía una vista unificada. BOOKLY centraliza ese proceso.
>
> La aplicación maneja tres roles. El administrador configura servicios, profesionales, usuarios y consulta la auditoría. El profesional administra su horario y ve sus turnos. El cliente explora el catálogo, consulta disponibilidad y realiza o cancela reservas.
>
> El valor principal es que la lógica de negocio no queda repartida entre formularios: existe una fuente central de información que mantiene consistentes la disponibilidad, las reservas y los permisos.”

Diapositiva 3 — Alcance funcional implementado — Ponente 1 — 1 minuto y 30 segundos

En pantalla

- Módulos por rol en una cuadrícula:
- **Administrador:** servicios, profesionales, agenda, usuarios, auditoría, perfil y seguridad.
- **Profesional:** agenda, horario semanal, turnos y perfil.
- **Cliente:** catálogo, detalle del servicio, disponibilidad, reservas y perfil.
- Accesos comunes: autenticación, MFA, cambio de contraseña, cierre de sesión.

Guion

> “En esta diapositiva podemos observar el alcance funcional alcanzado. Para el administrador ya existen la administración de servicios y profesionales, la consulta de la agenda, la gestión de usuarios y roles, la auditoría y las opciones de seguridad. Para el profesional están la agenda, la gestión del horario semanal y el seguimiento de turnos. El cliente dispone del catálogo, el detalle de cada servicio, la disponibilidad y la gestión de sus reservas.
>
> Todas las áreas comparten el mismo acceso seguro, incluyendo registro, inicio de sesión, autenticación en dos pasos para administradores, cambio de contraseña y cierre de sesión. Es decir, no tenemos pantallas aisladas: hay un flujo completo por rol y una lógica transversal de seguridad.”
>
> **Sugerencia de ejecución:** presentar este bloque mirando brevemente cada grupo de rol y detenerse solo en el proceso de reserva.

Diapositiva 4 — Avances destacados — Ponente 1 — 30 segundos

En pantalla

- **Aplicación Android instalable:** APK firmado y AAB generado.
- **Verificación en dispositivo:** Android 14.
- **Calidad:** 393 pruebas E2E aprobadas y 1 omitida.
- **UX móvil:** safe areas, controles táctiles de 44 px y navegación adaptable.

Guion

> “Los avances más relevantes se pueden resumir en cuatro frentes. Primero, la aplicación ya funciona como APK Android firmado y también genera el formato AAB. Segundo, fue instalada y verificada en un dispositivo con Android 14. Tercero, implementamos una suite automatizada con 393 pruebas aprobadas y una omitida. Y cuarto, cerramos los detalles que normalmente afectan la experiencia móvil: áreas seguras, teclado, botón atrás, navegación, tamaños táctiles y conectividad.
>
> El siguiente ponente explicará cómo se coordina la lógica de negocio detrás de estas pantallas.”

Diapositiva 5 — Lógica de negocio: la reserva como proceso central — Ponente 2 — 1 minuto

En pantalla

Diagrama horizontal: **Horario del profesional → Disponibilidad → Reserva → Agenda / auditoría**

Guion

> “La lógica de negocio puede entenderse a partir de una decisión principal: una reserva nunca es simplemente un registro con fecha y hora. Primero debe existir un horario vigente del profesional. Sobre ese horario, el sistema calcula la disponibilidad de un servicio concreto, descuenta las reservas existentes y bloquea los intervalos que no cumplen las reglas.
>
> Cuando el cliente selecciona un espacio disponible, el sistema crea una reserva asociada al servicio, al profesional y al cliente. Esa reserva actualiza la agenda de las partes involucradas y queda reflejada en la auditoría. Si el cliente cancela, el espacio vuelve a quedar disponible de acuerdo con las reglas del sistema.”

Diapositiva 6 — Reglas de negocio de la API — Ponente 2 — 1 minuto y 30 segundos

En pantalla

Cuatro tarjetas: 1. **Disponibilidad real** 2. **Conflicto controlado** 3. **Propiedad de la reserva** 4. **Auditoría y trazabilidad**

Guion

> “La API concentra las reglas que protegen la operación. En primer lugar, la disponibilidad se calcula combinando el horario semanal del profesional, la duración del servicio y las reservas ya ocupadas. De esta forma, la pantalla no inventa horas disponibles.
>
> En segundo lugar, cuando una reserva entra en conflicto —por ejemplo, porque otra persona tomó el mismo horario en ese momento— el sistema rechaza la operación y la interfaz informa que debe elegir otro espacio.
>
> En tercer lugar, cada reserva se encuentra vinculada a sus participantes. Cada rol consulta únicamente las reservas que le corresponden, y las acciones sensibles están protegidas.
>
> Finalmente, las operaciones importantes quedan registradas para que el administrador pueda auditar qué ocurrió, quién lo hizo y sobre qué recurso.”
>
> **Idea clave para este bloque:** “La API no solo guarda datos; aplica las reglas que mantienen coherente la agenda.”

Diapositiva 7 — Cómo la API comunica el resultado — Ponente 2 — 1 minuto

En pantalla

- Solicitud del frontend + validación de sesión
- Procesamiento de reglas
- Respuesta con datos o motivo de rechazo

- Diagrama de estados: **Pendiente → Confirmada → Cancelada**

Guion

> “Desde el punto de vista del frontend, el intercambio con la API sigue siempre el mismo camino. La aplicación envía la acción del usuario junto con la sesión vigente. El backend aplica las reglas de negocio y devuelve un resultado comprensible.
>
> Cuando la operación es exitosa, la respuesta contiene la información actualizada y la interfaz refresca la vista afectada. Cuando no es posible realizarla, la API explica el motivo, por ejemplo, horario ocupado, conflicto de horario, acceso no autorizado o cuenta bloqueada. El frontend traduce ese resultado a mensajes comprensibles, evita repetir una acción inválida y mantiene la interfaz sincronizada.
>
> En el caso de una reserva, los estados posibles son pendiente, confirmada y cancelada. La interfaz refleja esta transición, pero es la API quien protege la transición y sus consecuencias.”

Diapositiva 8 — Autenticación, permisos y manejo de sesión — Ponente 2 — 30 segundos

En pantalla

- Inicio de sesión → validación → acceso según rol
- MFA para cuentas administrativas
- Sesión persistente en el dispositivo
- Cierre seguro ante sesión vencida

Guion

> “El acceso también sigue una lógica centralizada. Primero se validan las credenciales; después se determina el rol y se muestra la sección correspondiente. Para cuentas administrativas se exige un segundo factor, y para usuarios que ya enrolaron su dispositivo existe la opción de confiar en él.
>
> En el móvil, la sesión se conserva localmente para no pedir el ingreso repetidamente. Si el token deja de ser válido, el frontend limpia la sesión y dirige al usuario al acceso sin afectar los datos ya descargados. Además, se corrigió un detalle importante: un error de contraseña durante el cambio de contraseña ya no expulsa al usuario.”

Transition

> “Con esta lógica ya se entiende cómo opera el negocio. Ahora veremos cómo el frontend se adaptó al celular y cómo verificamos que la experiencia sea realmente móvil.”

Diapositiva 9 — Tecnologías, empaquetado y decisiones de diseño — Ponente 3 — 1 minuto y 30 segundos

En pantalla

- **Frontend:** SvelteKit 2 + Svelte 5 + TypeScript + SCSS + Vite.
- **Empaquetado móvil:** Capacitor 8 + Android Gradle.
- **Calidad:** Playwright + pruebas E2E.
- Flujo: build web → `npx cap sync android` → Gradle → APK firmado / AAB.
- Decisión: continuar con Svelte para reusar el frontend web y mantener consistencia.

Guion

> "Para explicar cómo llegamos a Android, es importante mencionar las tecnologías que usamos y, sobre todo, por qué tomamos esa decisión.
>
> El frontend está construido con SvelteKit 2 y Svelte 5, con TypeScript para el tipado, SCSS para el sistema visual y Vite como herramienta de construcción. Decidimos continuar con Svelte porque no queríamos empezar de cero el frontend ni duplicar el trabajo. De esa forma pudimos reutilizar los componentes, las rutas, la lógica de sesión y el diseño que ya manejamos en la versión web. Esta decisión también nos permitió mantener consistencia entre ambas versiones: los mismos componentes y las mismas reglas de negocio se presentan igual en el navegador y dentro del APK.
>
> Para el empaquetado móvil usamos Capacitor 8, que permite reutilizar ese frontend web dentro de un contenedor nativo Android. Sobre él se conectaron los plugins de Capacitor para el status bar, el teclado, la red, el splash y el botón atrás. Gradle se encarga del empaquetado final y de la firma.
>
> El proceso de construcción tiene tres pasos: primero se genera el build web estático; luego `npx cap sync android` copia ese resultado al proyecto Android; finalmente, Gradle produce el APK de depuración, el APK firmado y el AAB para Google Play.
>
> Para calidad usamos Playwright, que nos permitió automatizar las pruebas de los flujos funcionales, el responsive y la accesibilidad. En resumen, no buscamos reescribir la aplicación, sino extender la que ya existía con una capa móvil nativa y conservarla coherencia entre las dos plataformas."

Diapositiva 10 — Integración con el dispositivo — Ponente 3 — 1 minuto

En pantalla

- Barra de estado con identidad de marca.
- Teclado que no cubre los campos.
- Botón atrás de Android.
- Aviso sin conexión.
- Safe areas para muesca y gestos.

Guion

> “El trabajo móvil no consiste únicamente en generar el APK. Para que la aplicación se sienta nativa integramos Capacitor con cinco capacidades concretas.
>
> Primero, la barra de estado adopta la identidad visual de BOOKLY. Segundo, el teclado se redimensiona de manera nativa para no tapar los campos. Tercero, el botón físico o gestual de Android recorre el historial de la aplicación y solo permite salir cuando el usuario está en la pantalla principal. Cuarto, se informa claramente cuando el dispositivo no tiene conexión. Y quinto, los componentes respetan las safe areas, de modo que la navegación no queda debajo de la muesca ni de la barra de gestos.
>
> También se empaquetaron las fuentes dentro de la aplicación para que la interfaz conserve su apariencia incluso sin acceso a servicios externos.”

Diapositiva 11 — Diseño móvil y experiencia de usuario — Ponente 3 — 1 minuto

En pantalla

- Navegación: **4 destinos + “Más”**.
- Objetivos táctiles de **44 × 44 px**.
- Contraste **WCAG AA**.
- Matriz de tamaños: 320, 360, 393, 412, 768 y 1440 px.

Guion

> “También ajustamos el diseño al uso táctil. En pantallas pequeñas, la barra del administrador ya no intentaba mostrar sus siete secciones. Ahora presenta cuatro destinos principales y un acceso llamado ‘Más’, donde permanecen las opciones secundarias.
>
> Todos los controles interactivos principales alcanzan al menos 44 por 44 píxeles, una medida adecuada para evitar errores de toque. Además, se corrigieron los colores que no cumplían el contraste AA y se verificó la interfaz en seis anchos representativos, desde teléfonos compactos hasta escritorio.
>
> El objetivo era que la aplicación se sintiera como una experiencia diseñada para el celular, y no como una versión reducida del sitio.”

Diapositiva 12 — Calidad y pruebas — Ponente 3 — 1 minuto

En pantalla

- **393 aprobadas / 1 omitida**
- Cobertura funcional por roles.
- **144** casos responsive.

- **202** casos de accesibilidad.
- APK verificado con `apksigner` y `zipalign`.

Guion

> "La calidad se validó de forma automatizada y reproducible. La suite final tiene 393 pruebas aprobadas y una omitida. No se trata de pruebas que siempre pasan: cada caso verifica un comportamiento observable.
>
> La cobertura funcional atraviesa los flujos de autenticación, administración, profesionales y clientes. A esto se suman 144 casos de comportamiento responsive y 202 casos de accesibilidad, incluyendo contraste, estructura, navegación por teclado, nombres accesibles y tamaño de las áreas táctiles.
>
> Además, el APK firmado se comprobó con las herramientas oficiales de Android para confirmar la firma y la alineación del paquete. También se verificó la instalación y el arranque en un dispositivo con Android 14."

Diapositiva 13 — Errores encontrados y decisiones de calidad — Ponente 3 — 1 minuto

En pantalla

- Tres tipos de hallazgo:
- **Frontend:** sesión expirada durante login; contraseña incorrecta cerraba sesión; auditoría con formato de fecha incorrecto.
- **Backend:** estado de cuenta bloqueada no devuelto; nombres de servicio y profesional ausentes en la respuesta.
- **Mitigación:** robustez en el cliente y documentación para el backend.

Guion

> "Las pruebas también sirvieron para encontrar errores reales. Por ejemplo, detectamos que una respuesta 401 de una petición antigua podía cerrar una sesión que acababa de iniciarse. También se corrigió el caso en que una contraseña actual incorrecta expulsaba al administrador, y se ajustó el formato de las fechas de auditoría que impedía mostrar resultados.
>
> También identificamos inconsistencias que corresponden al backend, como la ausencia de ciertos códigos de bloqueo y de nombres de servicio o profesional en la respuesta de reservas. Esas se documentaron y, cuando fue posible, el frontend las toleró para que la experiencia no se interrumpiera.
>
> Este balance es importante: la automatización no solo demuestra que las cosas funcionan; también revela dónde debe mejorar el sistema completo."

Diapositiva 14 — Estado actual y próximos pasos — Ponente 3 — 30 segundos

En pantalla

Completado

- APK firmado + AAB.
- Verificación en Android 14.
- Flujos principales por rol.
- Responsive y accesibilidad.
- 393 pruebas E2E en verde.

Próximos pasos

- Publicación en Google Play y validación del proceso de distribución.
- Pruebas con usuarios reales y dispositivos de gamas distintas.
- Mejora de la cobertura contractual de la API y resolución de incidencias backend.
- Seguimiento de métricas de uso y calidad.

Guion

> "El estado actual es que tenemos el frontend móvil completo para los flujos principales, un APK firmado, el AAB, pruebas automatizadas y validación real en Android 14.
>
> Como próximos pasos debemos preparar la publicación en Google Play, realizar pruebas con usuarios reales y ampliar la variedad de dispositivos. También queremos cerrar las inconsistencias de la API señaladas por la suite y definir métricas de uso y calidad para orientar la siguiente iteración."

Diapositiva 15 — Conclusiones — Ponente 1 — 45 segundos

En pantalla

Tres ideas: 1. **Lógica centralizada:** la API protege disponibilidad, reservas, permisos y trazabilidad. 2. **Experiencia móvil real:** la aplicación funciona y fue validada como APK Android. 3. **Calidad verificable:** 393 pruebas aprobadas y acceso a los artefactos de distribución.

Guion

> "Para cerrar, destacamos tres ideas. Primero, la lógica de negocio está centralizada: disponibilidad, reservas, permisos y auditoría responden a reglas consistentes. Segundo, el resultado no es solamente un sitio web adaptado: es una aplicación Android que integra el dispositivo y fue verificada en un entorno real. Y tercero, la calidad es verificable, con 393 pruebas aprobadas, validación de accesibilidad y artefactos listos para continuar el proceso de distribución.
>
> Con esto culminamos la presentación. Gracias por su atención."

Preguntas frecuentes y respuestas breves

1. ¿La aplicación ya está en Play Store?

> “No. Ya generamos el AAB y verificamos el APK firmado, pero la publicación en Google Play requiere completar el proceso de distribución, la cuenta de desarrollador y las revisiones de la tienda.”

2. ¿La API se develops también en este proyecto?

> “El backend es un sistema independiente. En esta entrega del frontend nos concentramos en consumir sus reglas, mantener la coherencia de la interfaz y documentar las inconsistencias detectadas.”

3. ¿Cómo se evita que dos clientes reserven el mismo horario?

> “La disponibilidad se calcula a partir de los horarios y reservas existentes. Además, la creación de la reserva vuelve a validar esa condición. Si otra reserva se completó primero, la API rechaza la operación y la interfaz solicita elegir otro espacio.”

4. ¿Qué ventaja tiene usar Capacitor?

> “Permite reutilizar el frontend ya desarrollado y añadir capacidades nativas de Android sin duplicar la aplicación. También centraliza la construcción del APK y del AAB.”

5. ¿Por qué 393 pruebas y no 1000?

> “El plan original proposed una meta paramétrica de 1000, pero priorizamos casos que verifican comportamientos reales y fallan cuando ocurre una regresión. La suite final distribuye la cobertura en flujos funcionales, responsive y accesibilidad.”

6. ¿Qué ocurre si el dispositivo pierde conexión?

> “La aplicación informa que no hay conexión. En el alcance actual no se implementó una cola completa de operaciones sin conexión; es una mejora prioritaria para una siguiente iteración.”

7. ¿La información de la auditoría es visible para todos?

> “No. La auditoría pertenece al módulo administrativo. Los demás roles ven sus reservas o turnos según sus permisos.”

Acceso de demostración con rol administrador y código MFA

Desde el directorio `frontend`, usa:

```
npm run mfa:codigo -- --loop
```

El comando muestra el código TOTP vigente de `admin@bookly.local` y lo actualiza cada segundo. Los códigos rotan cada 30 segundos.

Para mostrar únicamente el código numérico:

```
npm run mfa:codigo -- --solo
```

Credenciales de la cuenta administrativa

Campo	Valor
Correo	`admin@bookly.local`
Contraseña	`Admin123!Pass`
Código MFA	El código de 6 dígitos generado por `npm run mfa:codigo`

Procedimiento para iniciar sesión

1. Abre la aplicación web en `http://127.0.0.1:5173/login` o el APK instalado en el celular.
2. Ejecuta `npm run mfa:codigo -- --loop` en otra terminal.
3. Ingresa `admin@bookly.local` y `Admin123!Pass`.
4. Cuando solicite el segundo factor, escribe el código de 6 dígitos que aparece en la terminal.

Si el comando no encuentra un MFA enrolado

Los tests E2E pueden deshabilitar el MFA del administrador. Para restaurarlo:

```
npm run mfa:enrolar
npm run mfa:codigo -- --loop
```

Alternativamente, se puede reinstate mediante el seed:

```
MFA_ADMIN=1 npm run seed:e2e
```

> El comando obtiene el secreto cifrado desde la base de datos local `backendcf-db`; por tanto, el backend y la base de datos deben estar levantados. Esta credencial es exclusivamente para el entorno local de demostración.

Checklist antes de presentar

- [ ] Confirmar los nombres de los tres ponentes.
- [ ] Completar la fecha o el periodo académico en la portada.
- [ ] Sustituir “393 pruebas E2E-approved” por “393 pruebas aprobadas” en la diapositiva 4.
- [ ] Verificar que el número final de pruebas sea 393 aprobadas y 1 omitida.
- [ ] Mostrar una captura real de la aplicación en Android en la diapositiva 9 o 10.
- [ ] Verificar que el APK y el AAB estén disponibles para una demostración.
- [ ] Preparar el APK en un emulador o dispositivo con la API disponible.
- [ ] Ensayar una vez con cronómetro: objetivo de 9:30–10:00 minutos.
- [ ] Compartir las diapositivas con tiempo para preguntas.
- [ ] Eliminar de la presentación los fragmentos en otro idioma o marcadores de borrador.